SCALECART · DISTRIBUTED ORDER PROCESSING PLATFORM

Interview Questions & Answers

Parts 1–6 · 55 questions with full answers, based on what was actually built

Repository: github.com/MayurGaikwad57/Scale-Cart-Learning
Covers milestones M1–M5 (built and merged) and M6 (planned design) · November 2026

Contents

Part 1 — M1 Foundation (architecture, database design)	Q1–Q8
Part 2 — M2 Backend Core (auth, products, inventory, cart, orders, transactions)	Q9–Q18
Part 3 — M3 Frontend Core (Angular 21)	Q19–Q27
Part 4 — M4 Performance (own Redis "ScaleKV", caching, rate limiting, idempotency)	Q28–Q39
Part 5 — M5 Kafka (own Kafka "ScaleLog", events, consumer groups)	Q40–Q51
Part 6 — M6 Microservices (planned — not built yet)	Q52–Q55

How to use this document: read the question, answer it out loud first, then compare with the written answer. Parts 1–3 were written from what was built (the plan's "challenge" step had been skipped for them); Parts 4–5 include the question lists from `docs/06-performance.md` and `docs/07-events.md` word for word, plus the example questions from `project-plan.md` §25; Part 6 answers describe the planned design.

PART 1 · MILESTONE M1

Foundation

Architecture and database design

1 What is ScaleCart, in one minute?

An e-commerce order-processing platform I built to learn distributed systems step by step. It started as a **modular monolith**: one Node/Express + TypeScript backend with clear modules (users, products, inventory, cart, orders, payments) on PostgreSQL, plus an Angular 21 frontend. Each phase adds one production concern: caching and rate limiting (M4), event streaming (M5), then microservices, reliability patterns, scaling and observability. Under a no-install constraint, I built my own Redis-like store (ScaleKV) and my own Kafka-like broker (ScaleLog).

2 Why start with a modular monolith instead of microservices?

Microservices add network failures, distributed transactions, deployment and debugging cost from day one. A monolith gives one database transaction for checkout, simple deployment and fast refactoring. *Modular* means each module has its own routes, service and repository, and modules talk through service functions rather than each other's tables. So when a module needs to become a service later (payments in M6), the boundary already exists. You split when you have a reason, such as independent scaling, team ownership or failure isolation, not by default.

3 Why store money as integer cents instead of a decimal/float?

Floating point can't represent values like 0.1 exactly: 0.1 + 0.2 ≠ 0.3, so totals drift. Integer cents (`price_cents INT`) make every sum exact, and formatting to ₹ happens only at display time. (DECIMAL in the database also works, but integers are simpler to move between DB, JSON and JavaScript without precision loss.)

4 Why UUIDs as primary keys instead of auto-increment integers?

UUIDs can be generated anywhere (any service, any shard) without asking a central counter, so they survive moving data across services or shards later. They also don't leak business volume: order #1042 tells a competitor how many orders you have. The cost is bigger indexes and random insert order (B-tree page splits). Time-ordered UUIDs (v7) reduce that.

5 Which indexes did you create and why?

Each index exists for a specific query:

- `products (category, price_cents)` for the catalog filter by category plus price sort/range.
- `orders (user_id, created_at DESC)` for "my orders, newest first", so the database reads the index in order instead of sorting.
- `UNIQUE cart_items (cart_id, product_id)`: one line per product per cart, and it doubles as the index for "items of a cart".
- `UNIQUE users(email)`, `UNIQUE products(sku)` for lookups and duplicate prevention.
- `order_items(order_id)`, `payments(order_id)` for joining an order to its lines and payments.

6 What constraints protect data integrity in the database itself?

Rules the application could forget are enforced by PostgreSQL:

- `CHECK (available >= 0)` on inventory: stock can never go negative, even if the code has a bug.
- `CHECK (quantity > 0)` on cart items.
- Foreign keys between all tables.
- Unique constraints on email, SKU, cart line, and `(user_id, idempotency_key)` on orders.

The database is the last line of defence.

7 Why is inventory a separate table from products?

Products are read-heavy catalog data; inventory is write-heavy, contended data (every checkout locks and updates it). Separating them means a checkout's row lock on stock doesn't block reads or edits of product descriptions. It also prepares an independent Inventory service later. The table also has `reserved` (stock held during payment) and `version` (for optimistic locking).

8 What did you NOT finish in M1, honestly?

The Docker setup (`docker-compose.yml`) was written but never tested, because Docker can't be installed on the development machine. I run PostgreSQL as a local service instead. In an interview it's better to say this than to pretend it was verified.

PART 2 · MILESTONE M2

Backend Core

Auth, products, inventory, cart, orders, transactions

9 How does authentication work?

On register, the password is hashed with **bcrypt** (slow, salted hashing, so a leaked database can't be reversed quickly) and stored as `password_hash`. On login, bcrypt compares the hash and the server signs a **JWT** containing the user id (`sub`) and role, with an expiry. The client sends `Authorization: Bearer <token>` on every request, and the `authenticate` middleware verifies the signature and attaches `req.user`. `requireRole("ADMIN")` guards admin routes. JWTs are stateless (no session lookup), but you can't revoke one before it expires without extra machinery (short expiry + refresh tokens, or a deny-list).

10 Walk me through checkout. Why is it one transaction?

Inside one database transaction:

1. Lock the user's cart row (`SELECT … FOR UPDATE`).
2. Read the cart and reject an empty cart or inactive products.
3. Lock the stock rows (`FOR UPDATE … ORDER BY product_id`).
4. Check availability.
5. Move stock from `available` to `reserved`.
6. Create the order with a **price snapshot** per line.
7. Charge the (simulated) payment.
8. On success, confirm the order, turn the reservation into a sale and empty the cart. On decline, **compensate**: give the reserved stock back, mark the order CANCELLED, keep the cart.

Any error rolls back everything. You can never end up with an order but no stock change, or stock taken but no order.

11 Two customers buy the last item at the same moment. What happens?

Both transactions reach `SELECT … FROM inventory … FOR UPDATE`. PostgreSQL gives the row lock to one of them; the other **waits**. The first sees `available = 1`, reserves it and commits. Only then does the second get the lock, and it reads the **current** value `0` and fails with `insufficient_stock`. Without the lock, both could read `1` (a race condition) and oversell. The seeded "Limited Edition Smartwatch" with stock 1 exists to demonstrate this.

12 Why `ORDER BY product_id` when locking stock rows?

To prevent **deadlocks**. If cart A locks product 1 then 2, while cart B locks 2 then 1, each can hold one lock and wait forever for the other. If every transaction always locks rows in the same global order (sorted by id), a cycle of waiting is impossible.

13 Pessimistic vs optimistic locking — which did you use and why?

Pessimistic (`SELECT … FOR UPDATE`): lock first, then read and modify; others wait. **Optimistic**: read without locking, then update with `WHERE version = :readVersion`; if zero rows changed, someone else won, so retry. I used pessimistic for checkout because stock on a popular item is a **high-contention** row, where optimistic locking would cause many failed retries. Optimistic suits low contention (e.g. two admins rarely editing the same product). The `inventory.version` column is there for that comparison.

14 Why snapshot the price on each order item?

Prices change. If an order only referenced the product, raising the price tomorrow would rewrite last month's order totals and invoices. `order_items.unit_price_cents` freezes what the customer actually paid. History must be immutable.

15 Why soft-delete products instead of deleting rows?

Order items reference products through foreign keys, and past orders must stay readable. A hard delete would either fail (FK violation) or cascade and destroy order history. `active = false` hides the product from customers (it returns 404 for them) while admins still see it, and old orders still show its name.

16 Why does another user's order return 404 instead of 403?

`403 Forbidden` confirms the order **exists**, which leaks information (an attacker can enumerate valid order IDs). The query filters by `id AND user_id`, so someone else's order is simply "not found". This is a common defence against insecure direct object references (IDOR).

17 How do you validate input and handle errors consistently?

Every request body is validated with **zod** schemas in a `validateBody` middleware. Invalid input becomes `400 validation_failed` with per-field details. Expected failures throw an `HttpError(status, code, message)`. A single error-handler middleware turns everything into the same JSON shape (`{ error, message?, details? }`), and unexpected errors are logged but returned as `500 internal_error` without leaking internals. Config is also validated at boot (`env.ts`), so the app fails fast with a clear message.

18 How did you test the concurrency logic?

Integration tests run against the real PostgreSQL through the HTTP API (supertest). They include concurrent checkouts racing for the last unit, and assert that exactly one wins and stock never goes negative. Each test run tags its data with a unique prefix and cleans up only what it created. I also **mutation-checked** the tests: deliberately removing the lock or the check makes a test fail. That proves the tests actually detect the bug.

PART 3 · MILESTONE M3

Frontend Core

Angular 21: standalone, zoneless, signals

19 What is "modern Angular" in this project?

- **Standalone components**: no NgModules.
- **Zoneless change detection**: no zone.js; Angular updates the view when signals change.
- **Signals** for state.
- **New control flow** (`@if`, `@for`, `@let`).
- **Lazy-loaded routes** with `loadComponent`, so every page's code downloads only on first visit.
- `OnPush` on every component.
- **Route params bound as component inputs** (`withComponentInputBinding`).

20 Signals vs RxJS — how did you decide which to use where?

Signals are for *state*: synchronous values the UI reads (logged-in user, cart, notifications), with `computed()` for derived values (cart count and total can never disagree with the items). **RxJS** is for *async streams over time*: HTTP calls, debouncing the search box (`debounceTime`), cancelling stale requests (`switchMap`), polling with `timer`. The two meet through `toSignal()`. The rule from the plan: don't use signals just because they're new; pick per piece of state.

21 Where does each piece of state live?

- Logged-in user and cart: **signals in root services**, because the navbar, product pages, cart and checkout all need them at once.
- Reloading the cart on login/logout: an `effect()`.
- Form inputs: Reactive Forms (validation, touched/dirty).
- Loading/error flags, menus: local component signals.
- Session across reloads: `localStorage` + `/auth/me` at startup.

22 How does the HTTP interceptor work?

On every request to our API (and only our API, never third parties) it adds `Authorization: Bearer <token>`. If the API answers `401` on a protected call, the session is dead, so it logs out and redirects to `/login?returnUrl=…`. The login and register calls themselves are excluded, because a `401` there means "wrong password", not "expired session".

23 Are route guards security?

No. `authGuard`, `adminGuard` and `guestGuard` are **UX only**: they stop people from seeing pages they can't use. Anyone can call the API directly, so **the API enforces authorization** (`authenticate`, `requireRole`). Never trust the client.

24 What is an open redirect, and how did you prevent it?

After login we send the user back to `returnUrl`. If an attacker crafts `…/login?returnUrl=https://evil.com`, a naive app would forward a freshly logged-in user to a phishing site. `safeReturnUrl()` only allows in-app paths starting with a single `/`, and rejects absolute URLs and protocol-relative `//evil.com`. It's unit-tested.

25 Storing the JWT in localStorage — what's the trade-off?

It's simple and survives reloads, but any JavaScript running on the page can read it, so an **XSS** bug could steal the token. The hardened alternative is an **httpOnly, Secure, SameSite cookie** (JavaScript can't read it), which then needs CSRF protection. For a learning project I chose simplicity and documented the trade-off.

26 How does the product search avoid flooding the API while the user types?

Search, filter and page live in signals. A `computed()` builds the query, and RxJS adds `debounceTime` (wait until typing pauses) and `switchMap` (cancel the previous in-flight request when a new query starts, so an old slow response can't overwrite a newer one). The result is exposed via `toSignal`.

27 How did you verify the UI really works?

Unit tests for services and helpers (vitest), plus browser journeys driven in real Edge (headless, via puppeteer-core): login, add to cart, search, checkout declined and successful, order history, logout guard, admin product creation. I took screenshots of every page at desktop width and at 390px phone width. Testing on a phone width caught a real bug later (a notification dropdown cut off on the left that a "no horizontal scroll" check couldn't see).

PART 4 · MILESTONE M4

Performance

Own Redis “ScaleKV”, caching, rate limiting, idempotency

From docs/06-performance.md

28 Why must a cache or rate limiter be a separate process when you run several API instances?

In production several copies of the API run behind a load balancer. A `Map` inside one process is invisible to the others:

- **Cache:** each instance has its own copy, and an invalidation reaches only the instance that handled the write, so the others keep serving stale data.
- **Rate limiter:** each instance counts separately, so a client spread over N instances gets N× the limit.

Shared state has to live outside the app processes, which is exactly why Redis exists. That’s why ScaleKV is its own process that every API instance talks to over TCP.

29 Redis is single-threaded. Why is that fast, and why does it make `INCR` safe?

Each command is tiny in-memory work (microseconds), so the bottleneck is the network, not the CPU. A single thread means no locks, no lock contention and no context switches. And because commands run **one at a time to completion**, `INCR` (read, add 1, write) can never interleave with another client’s `INCR`. That makes it atomic for free. ScaleKV works the same way: Node runs JavaScript on one thread and `execute()` never awaits mid-command.

30 Walk through cache-aside. Why delete after the DB commit and not before? What race remains?

- **Read:** look in the cache. On a HIT, return it. On a MISS, read PostgreSQL, store the result with a TTL, return it.
- **Write:** update PostgreSQL first, *then* invalidate.

If you invalidate **before** the commit, a reader can slip in between, read the *old* row (not yet committed) and cache it again, and the stale value lives for the full TTL. A race remains even with delete-after-commit: a reader misses and reads the old row; the writer commits and deletes; *then* the slow reader stores the old row. My fix (found in code review) is **versioned keys**. Every product has a version counter (`product:ver:<id>`), the cache key includes it (`product:<id>:v<n>`), and a write bumps the counter. The slow reader’s late write lands on an old version that nobody reads anymore. TTL bounds any remaining staleness.

31 Explain stampede vs avalanche vs penetration and one fix for each.

- **Stampede (thundering herd):** a hot key expires and hundreds of requests miss at once and all hit the database. *Fix:* **single-flight**: concurrent misses for the same key share one database query (across instances: a short `SET lock NX PX` mutex, or refresh before expiry).
- **Avalanche:** many keys created together expire together, and the database gets the whole load at once. *Fix:* **TTL jitter** (±10% random), so expiries spread out.
- **Penetration:** requests for IDs that don’t exist always miss and always hit the database (often an attack). *Fix:* **negative caching**: cache “not found” briefly (15s), plus validating the UUID format before any lookup (at scale: a Bloom filter).

32 Fixed window vs sliding window counter vs token bucket: which would you pick for login, and for a public API?

- **Fixed window:** one counter per minute. Cheap, but allows a **2× burst at the boundary** (100 requests at 12:00:59 and 100 more at 12:01:00).
- **Sliding window log:** a timestamp per request. Exact, but memory grows with the limit.
- **Sliding window counter (what I built):** `estimate = previous × (1 − elapsed fraction) + current`. Two small counters, no boundary burst, only needs `INCR` / `GET`.
- **Token bucket:** a refill rate plus a capacity. Allows controlled bursts and needs an atomic read-modify-write (a Lua script in Redis).

For **login**, use a strict per-IP/per-account sliding window with no bursts (brute force is the threat), and possibly **fail closed** at a bank. For a **public API**, a token bucket lets well-behaved clients burst a little while capping the average. Mine **fails open**: if ScaleKV is down, requests are allowed, because a cache outage shouldn't become a full site outage.

33 A client retries `POST /orders` after a timeout. How do you guarantee one order? What if Redis lost the key?

The client sends an `Idempotency-Key` (one per purchase attempt). There are two layers:

1. **ScaleKV (fast layer):** `SET idem:order:<user>:<key> {running, fingerprint} NX PX 60s`.
   - Another request is still running with that key: **409** `request_in_progress`.
   - The first request finished: **replay** the stored response with `Idempotent-Replayed: true`.
   - Same key with a different body (SHA-256 fingerprint): **422** `idempotency_key_reused`.
   - The handler failed: delete the record so the key can be retried.
2. **PostgreSQL (the guarantee):** `UNIQUE (user_id, idempotency_key)` on orders plus the stored fingerprint.

If ScaleKV lost the key (restart, eviction, outage), the database still protects you. Inside the checkout transaction, **after locking the user's cart row**, we look for an order with that key ("lock, then check", so no race), and the unique constraint is the final backstop. The frontend keeps the same key on unknown outcomes (network error, 5xx, 409 in progress) and retries automatically, which is safe only because of the key.

34 Your client library times out a command but the reply arrives later. What can go wrong?

On a **pipelined** connection, replies are matched to commands purely by order (FIFO). If the timed-out command's slot is removed from the queue, its *late* reply gets handed to the **next** command. That command returns someone else's data, which is a silent, nasty bug. The fix: keep the slot but mark it settled, and drop its late reply when it arrives. A test reproduces head-of-line blocking to prove it. Separately, a **circuit breaker** treats 3 timeouts in a row as "server frozen": it drops the connection, fails fast and reconnects with backoff.

From project-plan.md §25, "After Redis"

35 Why Redis?

For shared, fast, in-memory state across all API instances: a cache in front of PostgreSQL (fewer and faster reads), rate-limit counters, and idempotency records. It gives atomic operations (`INCR`, `SET NX`), TTLs, and eviction when memory is full. In ScaleCart I couldn't install Redis, so I built ScaleKV, which speaks Redis's real RESP protocol. Switching to real Redis would only mean changing `KV_URL`.

36 Why Cache Aside?

The application controls caching explicitly. Only data that's actually requested gets cached (lazy loading), the cache can be down without breaking anything (we just read the database: `X-Cache: BYPASS`), and invalidation stays in our code next to the writes. Alternatives: **write-through** (write to cache and DB together; always warm, but caches data nobody reads), **write-behind** (write to cache, flush to DB later; fast, but risks data loss), **read-through** (the cache loads from the DB itself; needs cache-side loaders).

37 What happens when Redis goes down?

In ScaleCart the app **degrades instead of failing**:

- Commands fail fast (no waiting for timeouts).
- Product reads go straight to PostgreSQL (`BYPASS`, slower but correct).
- Rate limits **fail open**.
- Idempotency falls back to the database layer.
- The client reconnects in the background with exponential backoff (100ms → 5s), and the admin System page shows "Down".

I demonstrated this live by killing ScaleKV mid-use: the shop kept working and reconnected by itself when ScaleKV came back.

38 What happens during cache stampede?

A popular key expires, every concurrent request misses at the same moment, and they all query the database together. Database load spikes, latency rises, and timeouts can cascade. Mitigations: single-flight (I implemented it per process), a distributed lock so only one instance recomputes, early/probabilistic refresh before the TTL ends, and TTL jitter so keys don't all expire at once.

39 How would you invalidate the cache?

- **Single items:** after the database commit, invalidate the product's key. I bump a per-product version counter (versioned keys) so a late stale write is unreachable.
- **Many keys at once** (all catalog list pages, one per filter/sort/page combination): never `KEYS products:*` (O(N), and it blocks Redis's single thread). Instead `INCR products:ver`; every list key includes the version, so all old pages become unreachable at once and expire via their TTL.
- **Checkout** refreshes only the sold products' pages, so the list cache keeps its hit rate under steady orders (trade-off: list-page stock may lag up to 30s, while the product page and checkout are always exact).
- **At scale:** CDC-driven invalidation (M8).

PART 5 · MILESTONE M5

Kafka

Own Kafka "ScaleLog", events, consumer groups

From docs/07-events.md

40 Kafka vs a queue (RabbitMQ/SQS): what changes when consuming doesn't delete?

In a queue, a consumed and acknowledged message is **removed**, and one message goes to one consumer. In Kafka the topic is an **append-only log**: reading doesn't delete, and each **consumer group** keeps its own position (**offset**). So:

- Many independent groups can each read every event (notification, inventory and analytics all see every `OrderCreated`).
- You can **replay** history by moving an offset.
- A new consumer can be added later and process old events.
- Progress is a single number per partition instead of per-message acks.

Retention is time- or size-based, not consumption-based.

41 How does Kafka keep events of one order in order, and what does that cost?

Order is only guaranteed **within a partition**. The producer uses a **key** (the order ID), and `hash(key) % partitions` sends all events of that order to the same partition. One partition is processed by one group member, sequentially, so per-key order holds. The costs:

- There's **no global order** across partitions.
- A group's parallelism is capped by the partition count (4 members on 3 partitions means one is idle).
- A **hot key** overloads its partition.
- Changing the partition count remaps keys, which breaks ordering during the change.

42 A consumer crashes after writing to the DB but before committing. What happens, and how do you make it safe?

The offset wasn't committed, so after the rebalance the event is **delivered again** (to the restarted consumer or another member). That's **at-least-once** delivery: commit *after* handling. (Committing before handling would be at-most-once, where a crash loses the event.) To make it safe, consumers must be **idempotent**. In ScaleCart, each handler runs inside `once(group, eventId, …)`, which inserts `(consumer_group, event_id)` into `processed_events` **in the same database transaction** as the side effect. Either both commit or both roll back, and a redelivery hits the primary key and is skipped. That turns at-least-once *delivery* into exactly-once *effect*. A tested subtlety from code review: only a conflict on that marker means "duplicate"; any other error must be retried, or the event is silently lost.

43 Explain a rebalance. What is a zombie consumer and how is it stopped?

Group members send **heartbeats** (every 1s in ScaleLog). When a member joins, leaves (graceful LEAVE) or misses heartbeats for the **session timeout** (10s), the broker reassigns the partitions (range assignment) and bumps the group's **generation** number. Measured live: after a hard kill, the API took over in 10.3s. A **zombie** is a member that was declared dead (e.g. it froze for a GC pause or a network blip) but is still running and tries to commit for a partition that now belongs to someone else. **Fencing** stops it: every commit carries the member's generation, and the broker rejects a stale generation (`ILLEGAL_GENERATION`) or a partition the member doesn't own (`NOT_ASSIGNED`). The zombie then re-reads its assignment, and its uncommitted event is redone by the new owner (a duplicate, which idempotency absorbs).

44 One event always fails. What happens without a DLQ, and what does a DLQ cost you?

Without a DLQ, the consumer retries that **poison message** forever. Since a partition is processed in order, everything behind it is stuck: **head-of-line blocking**. Lag grows and customers stop getting notifications. With a DLQ, after N retries with exponential backoff (ScaleLog: 3 attempts, 200ms/400ms) the record is copied to `orders.<group>.dlq` with the error, the attempt count and its original location; the offset is committed and the partition keeps flowing. If even the DLQ write fails, we don't commit, so nothing is lost. The **cost** is ordering: while an event waits in the DLQ, later events for the same order can overtake it, so a replay can apply things out of order. There's also operational work: someone must inspect and **replay** (I built an admin Replay button, plus a "chaos switch" to demonstrate it).

45 Your order is saved but the event wasn't published. Why can this happen and how do you fix it?

It's the **dual-write problem**. Checkout writes the order to PostgreSQL and then sends the event to the broker: two systems with no shared transaction. If the broker is down, or the process dies between the two writes, the order exists but its event never does, so there's no notification and no analytics. ScaleCart doesn't fail the checkout for this; it **counts** missed events and shows a warning on the admin Events page. The fixes:

- **Transactional outbox (M7):** write the event into an `outbox` table in the *same* transaction as the order; a relay process publishes from the outbox and marks rows as sent (at-least-once again, so consumers stay idempotent).
- **CDC (M8):** stream the database's own change log (WAL) into the broker.

46 What does "consumer lag" tell you and when do you alert on it?

Lag = log end offset − committed offset, per partition: events written but not yet processed by that group. Some lag that stays steady is normal. **Lag that keeps growing** means consumers can't keep up (too slow, crashing, stuck on a poison message, or too few partitions/members). Alert on sustained growth or on lag measured in time (e.g. "notifications more than 2 minutes behind"), not on a single spike. The admin Events page shows lag per group and partition.

From project-plan.md §25, "After Kafka"

47 What happens when a consumer crashes?

Its heartbeats stop. After the session timeout (10s) the broker removes it, rebalances, and gives its partitions to the remaining members (new generation). They resume from the **last committed offset**, so any event the crashed consumer handled but didn't commit is processed again. Nothing is lost (at-least-once), and duplicates are absorbed by idempotent handlers. A graceful stop (Ctrl+C) sends LEAVE, so the rebalance is immediate instead of waiting 10s.

48 What happens before offset commit?

The consumer fetches records from its position, runs the handler for one record (with retries; on final failure it writes the record to the DLQ), and only **then** commits `offset + 1`. Until that commit succeeds, the broker still considers the record unprocessed. If the consumer dies at any point before the commit, the record is redelivered. If the commit is rejected (fenced after a rebalance), the consumer drops its position and refreshes its assignment.

49 How do you handle duplicates?

Assume they will happen: redelivery after a crash or rebalance, producer retries, and DLQ replays all cause them. Every event carries a unique `eventId`. Each consumer group records processed IDs in `processed_events`, in the same transaction as its work, so the second delivery is a no-op. Some effects are naturally idempotent (an upsert setting a value); others are not (`revenue += total`), and those *must* be guarded. I tested it by re-appending an already-processed event: still one notification, and revenue counted once.

50 What causes rebalancing?

- A member **joins** (a new worker starts; `npm run worker` triggers it).
- A member **leaves** gracefully (shutdown sends LEAVE).
- A member **misses heartbeats** past the session timeout (crash, freeze, long GC pause, network partition).
- In real Kafka also: a change in the topic's partitions, or consumers exceeding `max.poll.interval` (taking too long between polls).

Rebalances pause progress briefly and can cause duplicates, so you avoid unnecessary ones (stable membership, sensible timeouts, cooperative rebalancing in real Kafka).

51 How do partitions affect scaling?

A partition is the unit of parallelism: within one group, each partition is consumed by exactly one member. So the **maximum useful consumers in a group = the number of partitions** (3 partitions means at most 3 busy members; a 4th sits idle, as a test shows). More partitions mean more parallelism and throughput, but also more files, more open handles, longer rebalances, and ordering only per key. Increasing partitions later changes key-to-partition mapping, so choose the count with growth in mind.

PART 6 · MILESTONE M6

Microservices

Planned — questions from project-plan.md §25, answered from the planned design

Note: M6 is not built yet. These four questions come from the project plan; the answers describe the design we intend to build.

52 Why did you extract Payment?

Payment is the best first candidate. It talks to an external provider (slow, unreliable, rate-limited), has its own failure modes and security/compliance concerns (card data, PCI scope), and benefits from independent scaling, deployment and isolation: a payment-provider outage shouldn't take down browsing. Our code already isolates it (`paymentsService.charge`), so the boundary is clear.

53 Why not extract everything?

Every extraction adds network calls that can fail, data duplicated across databases, distributed transactions (sagas instead of one SQL transaction), more deployments, harder debugging (needs tracing), and operational cost. Products, cart and users have no strong reason to be separate yet. You extract when there's a concrete driver (scaling, ownership, failure isolation, release cadence), not for its own sake. Splitting too early creates a "distributed monolith": all the costs, none of the benefits.

54 How do services communicate?

Two styles, chosen per interaction:

- **Synchronous** HTTP/REST (or gRPC) when the caller needs an answer *now* (e.g. checking a price), always with timeouts, retries and circuit breakers.
- **Asynchronous** events through the broker (our ScaleLog) when the caller only announces a fact ("OrderCreated", "PaymentSucceeded") and others react in their own time. This gives loose coupling and resilience to downtime.

An **API gateway** sits in front for the browser: one entry point, routing, authentication, rate limiting.

55 What happens when one service is unavailable?

It depends on the coupling:

- **Synchronous calls:** fail fast with a timeout, a circuit breaker that stops hammering a dead service, and a fallback or graceful degradation (show cached data, or "payment pending").
- **Asynchronous events:** they simply wait in the log until the service comes back, then it catches up from its committed offset with no data lost. That's the same behaviour we already saw when a consumer group was down in M5.

The design goal is that one service failing degrades one feature, not the whole shop.